Chapter 4: Experimental Setup

Created by	Aroosh Ahmad
Created time	May 18, 2026 9:24 AM
Last edited by	Aroosh Ahmad
Last updated time	May 25, 2026 5:12 AM

4.1 Overview

This chapter describes the experimental setup used to evaluate batch scheduling strategies for instruction fine-tuning. The purpose of the experiments is to compare standard random batching with semantic grouping, two-phase curriculum scheduling, and length-based curriculum scheduling under a controlled fine-tuning environment.

The experiments are designed to investigate whether mini-batch construction and training order influence optimization behavior, generalization, and generation quality. To ensure a fair comparison, the same base model, fine-tuning method, dataset source, evaluation protocol, and multi-seed aggregation procedure are used across comparable experiments.

The experimental setup consists of the following components:

1. Preparation of Dolly instruction-following subsets.
2. Fine-tuning `google/flan-t5-small` using LoRA.
3. Implementation of random, grouped, curriculum, and length-based batch scheduling strategies.
4. Multi-seed evaluation using seeds 13, 21, and 42.
5. Evaluation using loss-based metrics and, where applicable, generation-based metrics.

4.2 Repository and Experiment Artifacts

The experimental implementation was organized through configuration files, training scripts, batching modules, evaluation scripts, and result artifacts. The inspected files included experiment configuration files from `exp_012` through `exp_032`, training modules, batching samplers, semantic grouping utilities, dataset processing scripts, FAISS indexing scripts, evaluation scripts, and reporting scripts. The repository also included experiment manifests, phase summaries, trainer states, adapter configurations, and aggregated CSV/JSON result reports.

The most relevant components were:

- `configs/experiments/exp_012_random_only_multiseed.yaml` through `configs/experiments/exp_032_hard_to_easy_length_longer_training_5k.yaml`
- `src/training/experiment_runner.py`
- `src/training/phase_runner.py`
- `src/training/trainer_factory.py`
- `src/batching/samplers.py`

- `src/batching/grouping.py`
- `src/batching/curriculum_sampler.py`
- `scripts/evaluation/evaluate_generation_quality.py`
- `reports/master/master_summary_table.csv`
- `reports/generation_eval_5k/generation_metrics_summary_table.csv`

This structure allowed each experiment to be traced from configuration to training output and final reporting.

4.3 Dataset Description

The experiments used subsets of the Databricks Dolly instruction-following dataset. The Dolly dataset contains instruction–response examples across multiple task types, making it suitable for evaluating supervised instruction fine-tuning.

Three dataset scales were used:

Dataset Variant	Train Size	Evaluation Size	Total Size	Purpose
Dolly small 1k	900	100	1,000	Pilot and controlled experiments
Dolly 3k	2,700	300	3,000	Intermediate scaling experiments
Dolly 5k	4,500	500	5,000	Main validation and curriculum experiments

For the 3k and 5k datasets, the upstream dataset was `databricks/databricks-dolly-15k`. The dataset was shuffled using seed 42, and subsets of 3,000 and 5,000 examples were selected. A 90/10 train/evaluation split was then created using `train_test_split(test_size=0.1, seed=42)`.

For the 1k dataset, the executed split was confirmed as 900 training examples and 100 evaluation examples. The 1k preprocessing was recovered from training notebooks, where the dataset was deterministically shuffled using seed 42, the first 100 examples were used for evaluation, and the remaining 900 examples were used for training.

4.4 Dataset Preprocessing

Each dataset example was represented as an instruction–response pair. The instruction was used as the model input, while the response was used as the target output for supervised fine-tuning.

For the 1k experiments, the prompt format used an instruction-style template:

```
### Instruction:
{instruction}

### Context:
{context}

### Response:
```

When no context was available, the context block was omitted:

```
### Instruction:
{instruction}

### Response:
```

The target output was the stripped response text. For 1k experiments, tokenization used `max_length=256`, `truncation=True`, and `padding=False`.

For 3k and 5k experiments, the input format was simpler:

```
{instruction}

Context: {context}
```

When no context was available, only the instruction was used:

```
{instruction}
```

For 3k and 5k experiments, both inputs and labels were tokenized using `max_length=512`, `truncation=True`, and `padding="max_length"`. The retained columns were `input_ids`, `attention_mask`, `labels`, and `raw_idx`.

The tokenizer was loaded using:

```
AutoTokenizer.from_pretrained("models/flan-t5-small")
```

The tokenizer configuration reports `T5Tokenizer` with a maximum model length of 512 tokens.

4.5 Base Model

The base model used in all experiments was `google/flan-t5-small`. The production training code loaded the model using `AutoModelForSeq2SeqLM`, while the local configuration identified the architecture as `T5ForConditionalGeneration`. The model is an encoder-decoder architecture suitable for sequence-to-sequence instruction fine-tuning.

The model setup is summarized below:

Component	Value
Base model	`google/flan-t5-small`
Local model name	`flan-t5-small`
Model class	`AutoModelForSeq2SeqLM`
Architecture	`T5ForConditionalGeneration`
Model type	Encoder-decoder
Tokenizer	`T5Tokenizer`

No explicit `torch_dtype`, `device_map`, quantization, or gradient-checkpointing settings were found in the final production training scripts.

4.6 LoRA Fine-Tuning Configuration

Fine-tuning was performed using Low-Rank Adaptation (LoRA). LoRA was selected because the study required repeated fine-tuning runs across multiple batching strategies, dataset sizes, and random seeds. Updating all model parameters for every experiment would have been computationally expensive, while LoRA enabled efficient comparison under practical hardware constraints.

The confirmed LoRA configuration is shown in Table 4.2.

Table 4.2: LoRA Configuration

Hyperparameter	Value
LoRA rank `r`	16
LoRA alpha	32
LoRA dropout	0.05
Target modules	`["q", "v"]`
Task type	`SEQ_2_SEQ_LM`
Bias	`none`
Trainable parameters	688,128
Total parameters	77,649,280
Trainable percentage	0.8862%

These values were confirmed from notebook output and executed adapter configuration files.

4.7 Training Configuration

The final scripted experiments used a consistent Hugging Face training configuration across comparable runs. The optimizer was AdamW Torch, with a linear learning-rate scheduler. Training was performed using per-device batch size 8 and gradient accumulation of 4, resulting in an effective training batch size of 32 per process.

Table 4.3: Training Configuration

Setting	Value
Optimizer	`AdamW_Torch`
Scheduler	Linear
Warmup steps	0
Per-device train batch size	8
Per-device eval batch size	8
Gradient accumulation steps	4
Effective train batch size	32
Weight decay	0.01
Logging steps	10
Evaluation strategy	Steps
Evaluation steps	50
Save strategy	Epoch
Save total limit	2
FP16	False
BF16	False
Max steps	-1
`predict_with_generate` during training	False

Training was organized into two phases. The phase learning rates were:

Phase	Learning Rate
Phase 1	0.00007
Phase 2	0.00005

The number of epochs varied by experiment block:

Experiment Block	Epochs per Phase
`exp_012` – `exp_015`	0.2
`exp_018` – `exp_031`	0.5
`exp_032`	1.0

These settings were confirmed from YAML configuration files and persisted training arguments.

4.8 Seed Handling

The experiment configurations used three seeds:

13, 21, 4213,\ 21,\ 42

13, 21, 42

These seeds were passed as sampler seeds to the custom samplers. The custom samplers used the seed to control batch ordering. However, persisted Hugging Face training arguments showed `seed=42`, and no explicit global `set_seed`, `torch.manual_seed`, or equivalent call was found in the final production scripts for experiments `exp_012` to `exp_032`.

This is important for interpretation. The multi-seed protocol primarily reflects variation in sampler ordering rather than necessarily full model-level random initialization. Therefore, results are interpreted conservatively, especially where numerical differences are very small.

4.9 Batch Scheduling Strategies

The experiments evaluated several batch scheduling strategies. These strategies were implemented through deterministic samplers inside a two-phase LoRA fine-tuning pipeline.

The main strategies were:

1. Random batching
2. Semantic grouped batching
3. Grouped → Random curriculum
4. Random → Grouped curriculum
5. Easy → Hard length curriculum
6. Hard → Easy length curriculum

Mixed batching was considered during exploratory analysis; however, no production mixed sampler or final controlled mixed-batching configuration was found in the final `configs/`, `src/`, or `scripts/` directories. Therefore, the final controlled comparison focuses on random, grouped, two-phase curriculum, and length-based curriculum strategies.

4.9.1 Random Batching

Random batching was implemented using `RandomBatchSampler`. The sampler shuffled all processed training indices using `random.Random(seed)` and emitted contiguous batches of size 8.

This strategy served as the primary baseline because random batching is the standard mini-batch construction method in most supervised fine-tuning pipelines.

4.9.2 Semantic Grouped Batching

Semantic grouped batching used an embedding-based nearest-neighbor approach. The semantic index bundle was loaded, and each processed training row was mapped to its corresponding raw row. Nearest neighbors were then retrieved, filtered to training-eligible rows, trimmed to `max_group_size=8`, and mapped back to processed indices.

The grouped sampler shuffled anchors, marked emitted samples in a global `seen` set, padded underfilled batches from remaining unseen examples, and performed a final leftover pass. This ensured that samples were not repeatedly emitted as part of multiple grouped batches.

The semantic grouping details are summarized in Table 4.4.

Table 4.4: Semantic Grouping Configuration

Dataset	Embedding Text	Embedding Model	FAISS Index	Similarity	Top-k	Batch Size
Dolly 1k	Instruction only	`all-MiniLM-L6-v2`	`IndexFlatIP`	Cosine via normalized inner product	32	8
Dolly 3k	Instruction + context	`all-MiniLM-L6-v2`	`IndexFlatIP`	Cosine via normalized inner product	8	8
Dolly 5k	Instruction + context	`all-MiniLM-L6-v2`	`IndexFlatIP`	Cosine via normalized inner product	8	8

The 1k experiments used instruction-only embeddings, while the 3k and 5k experiments used instruction-plus-context embeddings. In all cases, cosine similarity was implemented using normalized embeddings with inner product search through FAISS `IndexFlatIP`.

4.9.3 Two-Phase Curriculum Scheduling

Two-phase curriculum scheduling divided training into two phases. Phase 2 initialized from the adapter produced by Phase 1, allowing the second phase to continue training from the first phase instead of restarting from the base model.

Two curriculum directions were evaluated.

Random → Grouped

In the Random → Grouped strategy, Phase 1 used random batching and Phase 2 used semantic grouped batching. This strategy tests whether early diverse training followed by later semantic refinement improves learning behavior.

Grouped → Random

In the Grouped → Random strategy, Phase 1 used semantic grouped batching and Phase 2 used random batching. This reverse order acts as a control condition to test whether early semantic structure is preserved or diluted by later randomization.

4.9.4 Length-Based Curriculum Scheduling

Length-based curriculum was implemented using `CurriculumBatchSampler`. The sampler sorted examples according to a length-based score and emitted batches based on the selected direction.

The implemented length score was:

length score=len(input_ids)+count(labels≠−100)\text{length score} = \text{len(input_ids)} + \text{count(labels} \neq -100)

length score=len(input_ids)+count(labels=−100)

Two sorting directions were used:

Strategy	Sorting Direction	Interpretation
Easy → Hard Length	Ascending	Shorter/easier examples first
Hard → Easy Length	Descending	Longer/harder examples first

The length curriculum experiments were configured as follows:

Experiment	Strategy	Phase Structure
`exp_030`	Easy → Hard Length	Two phases, both easy-to-hard, 0.5 + 0.5 epochs
`exp_031`	Hard → Easy Length	Two phases, both hard-to-easy, 0.5 + 0.5 epochs
`exp_032`	Hard → Easy Length	Two phases, both hard-to-easy, 1.0 + 1.0 epochs

One implementation caveat was identified. In the processed Dolly 5k data, `input_ids` and `labels` were both fixed-length 512, and labels were padded with `0` rather than `-100`. As a result, the first 100 inspected training samples produced the same curriculum score of 1024.

This caveat should be acknowledged when interpreting length-based curriculum results.

4.10 Experiment Inventory

The experiments were organized into multiple groups, beginning with 1k pilot experiments and extending to 5k curriculum experiments. The main experiment groups are summarized in Table 4.5.

Table 4.5: Experiment Groups

Experiment IDs	Dataset Size	Methods	Training Duration
`exp_012` – `exp_015`	1k	Random, Grouped, Grouped → Random, Random → Grouped	0.2 epochs per phase
`exp_018` – `exp_021`	1k	Random, Grouped, Grouped → Random, Random → Grouped	0.5 epochs per phase
`exp_022` – `exp_025`	3k	Random, Grouped, Grouped → Random, Random → Grouped	0.5 epochs per phase
`exp_026` – `exp_029`	5k	Random, Grouped, Grouped → Random, Random → Grouped	0.5 epochs per phase
`exp_030` – `exp_031`	5k	Easy → Hard Length, Hard → Easy Length	0.5 epochs per phase
`exp_032`	5k	Hard → Easy Length	1.0 epoch per phase

The experiment inventory confirmed configuration files and output artifacts for all major runs from `exp_012` to `exp_032`.

4.11 Evaluation Setup

The evaluation procedure included loss-based evaluation during training and generation-based evaluation for selected 5k experiments.

Loss-based evaluation used the processed evaluation split stored with each dataset variant. The reporting pipeline computed:

Generalization Gap=Final Eval Loss−Final Train Loss\text{Generalization Gap} = \text{Final Eval Loss} - \text{Final Train Loss}

Generalization Gap=Final Eval Loss−Final Train Loss

ΔEval=Phase 2 Eval Loss−Phase 1 Eval Loss\Delta \text{Eval} = \text{Phase 2 Eval Loss} - \text{Phase 1 Eval Loss}

ΔEval=Phase 2 Eval Loss−Phase 1 Eval Loss

The generation evaluation script reconstructed the evaluation split from the raw training data using `train_test_split(test_size=0.1, seed=42)`. Inputs were formatted as either the instruction alone or as instruction plus context.

The generation configuration is summarized in Table 4.6.

Table 4.6: Generation Evaluation Configuration

Setting	Value
Generation batch size	8
Max input length	512
Max new tokens	128
`do_sample`	False
`num_beams`	Not found
`temperature`	Not found
`top_p`	Not found
ROUGE package	`evaluate.load("rouge")`
BERTScore package	`evaluate.load("bertscore")`
BERTScore language	`lang="en"`
5k evaluation samples	500

ROUGE was computed using `evaluate.load("rouge")`, and BERTScore was computed using `evaluate.load("bertscore")` with `lang="en"`. Metrics were computed per seed and then aggregated using mean and standard deviation.

Generation evaluation artifacts were found for `exp_026` to `exp_031`. No saved generation-evaluation artifacts were found for `exp_012` to `exp_025` or `exp_032`.

4.12 Hardware and Software Environment

The experiments were executed in GPU-enabled environments. Early notebook metadata indicated the use of an L4 GPU. Final scripted run manifests confirmed CUDA availability but did not record the exact GPU model.

The confirmed software environment is shown in Table 4.7.

Table 4.7: Software Environment

Component	Version
Python	3.10.20
PyTorch	2.5.1+cu121
Transformers	5.5.3
PEFT	0.18.1
Datasets	4.8.4
Sentence Transformers	5.4.0
FAISS CPU	1.13.2
CUDA available	True

The explicit CUDA runtime version was not found; only the PyTorch build tag `+cu121` was recorded.

4.13 Reproducibility and Known Uncertainties

Several measures were used to improve reproducibility:

1. All final experiments used the same base model.
2. LoRA configuration was kept fixed across comparable experiments.
3. Training arguments were consistent across experiment groups, except for planned epoch differences.
4. Batch scheduling strategies were implemented through explicit sampler classes.
5. Key experiments were run with seeds 13, 21, and 42.
6. Results were aggregated using mean and standard deviation.

However, several uncertainties remain:

- The exact raw provenance of the 1k Dolly subset is partially uncertain.
- A production `.py` preprocessing script for the 1k token cache was not found.
- No production mixed batching implementation was found.
- GPU model for scripted runs was not recorded.
- Explicit CUDA runtime version was not recorded.
- `num_beams`, `temperature`, and `top_p` were not found for generation evaluation.
- BERTScore model name was not explicitly recorded.
- Multi-seed variation primarily reflects sampler ordering because full global trainer/model RNG seeding was not explicitly found.

These uncertainties do not invalidate the experiments, but they should be acknowledged because the numerical differences between some methods are small. Therefore, interpretation of small improvements is kept conservative.

4.14 Summary

This chapter described the experimental setup used to evaluate batch scheduling strategies for instruction fine-tuning. The experiments fine-tuned `google/flan-t5-small` using LoRA with rank 16, alpha 32, dropout 0.05,

and target modules `q` and `v`. The experiments used Dolly 1k, 3k, and 5k subsets, with 90/10 train/evaluation splits. Training used AdamW Torch, a linear scheduler, batch size 8, gradient accumulation of 4, and two-phase learning rates of `7e-05` and `5e-05`.

Semantic grouping used `all-MiniLM-L6-v2` embeddings and FAISS `IndexFlatIP` nearest-neighbor retrieval. Curriculum experiments compared Random → Grouped, Grouped → Random, Easy → Hard Length, and Hard → Easy Length schedules. Evaluation included train loss, evaluation loss, generalization gap, phase-wise evaluation loss change, ROUGE, and BERTScore where generation evaluation was available.

The next chapter presents the experimental results and analyzes how the different batch scheduling strategies performed across dataset sizes, training durations, and evaluation metrics.